

---

# QuestCash

Monitoreo, firewall y balanceador de carga

---

## EP.03.01 — Entregable ED, Semana 3

*Que el sistema esté vigilado, protegido a nivel de red  
y sea tolerante a fallas*

---

|                     |                                                                              |
|---------------------|------------------------------------------------------------------------------|
| Proyecto integrador | QuestCash                                                                    |
| Materia             | Seguridad Informática                                                        |
| Equipo              | Proyecto QuestCash                                                           |
| Componentes         | questcash_web (Flask) · questcash_mobile (Expo)                              |
| Infraestructura     | Docker Compose · Nginx · PostgreSQL · Netdata                                |
| Producción          | Render (plan free), dominio <a href="https://onrender.com">.onrender.com</a> |
| Autor               | Armando Yael Contreras Bejarano                                              |
| Fecha               | 17 de agosto de 2026                                                         |

---

**Sobre las evidencias de este documento.** Todas las configuraciones que aparecen aquí (Nginx, Docker Compose, reglas de ufw, scripts de prueba) son **archivos reales del proyecto**, no ejemplos ilustrativos: están en `questcash_web/lab-semana3/` y se pueden ejecutar tal cual.

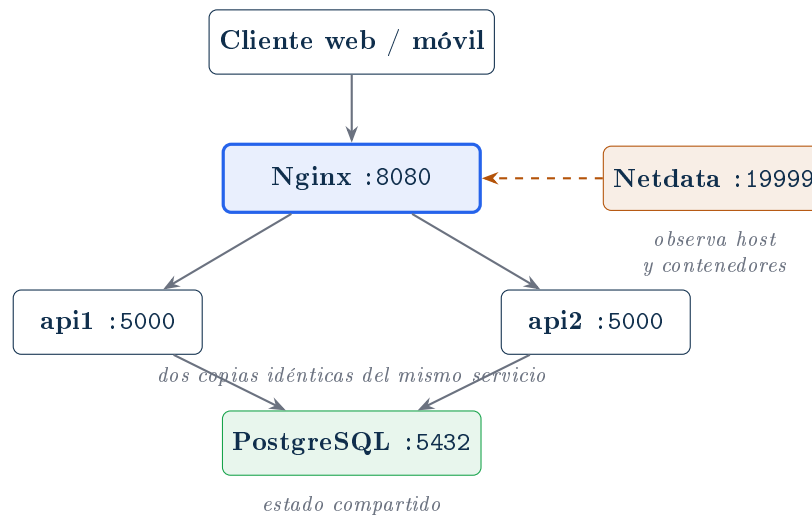
Las evidencias *visuales* —capturas del panel de monitoreo, del firewall y de la prueba de failover— aparecen como **recuadros vacíos marcados**, no como imágenes simuladas. Cada recuadro indica exactamente qué debe mostrar. Se sustituyen por la captura correspondiente al ejecutar el laboratorio.

## 1. Introducción

La actividad EP.03.01 pide incorporar tres mecanismos al proyecto integrador: **observabilidad** (saber qué está pasando en el sistema), **protección de red** (reducir la superficie expuesta) y **tolerancia a fallas** (que la caída de un componente no tumbe el servicio).

La solución mantiene la aplicación QuestCash detrás de un proxy Nginx, ejecuta dos instancias equivalentes del servicio y conserva la base de datos como un servicio separado y compartido. Así, la falla de una instancia no implica la caída del servicio.

### 1.1 Arquitectura objetivo



**Figura 1:** Topología del laboratorio. El único puerto publicado hacia fuera es el del balanceador.

**El móvil no se balancea.** La app de Expo es un cliente: consume `/api/v1`. El balanceador se coloca delante del servicio web/API para repartir las peticiones entre dos copias idénticas del backend. Balancear significa duplicar *el mismo* servicio, no repartir componentes distintos entre máquinas.

### 1.2 Por qué un laboratorio y no el entorno de producción

La aplicación pública de QuestCash está desplegada en **Render, plan free**. Render es un PaaS: administra el sistema operativo por nosotros. Eso tiene consecuencias directas sobre los tres puntos de esta semana.

| Requisito             | En Render (free)                                                                      | En el laboratorio                                                      |
|-----------------------|---------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| Firewall a nivel SO   | No hay acceso al sistema operativo ni SSH. Render aplica sus propias reglas de borde. | <code>ufw</code> real sobre Linux, con política <i>deny incoming</i> . |
| Dos instancias        | El plan free da una sola instancia, y además se suspende por inactividad.             | Dos réplicas idénticas ejecutándose simultáneamente.                   |
| Apagar una instancia  | No se puede: no hay segunda instancia que apagar.                                     | <code>docker stop qc_api2</code> con tráfico en vivo.                  |
| Monitoreo del sistema | Métricas básicas del panel de Render; sin acceso a <code>/proc</code> .               | Netdata con visibilidad de CPU, RAM, red, disco y contenedores.        |

**Tabla 1:** Por qué los tres puntos requieren un entorno con control del servidor.

Esto no es un rodeo: es la razón técnica por la que la práctica se monta sobre Docker Compose. Un PaaS *abstrae* justamente los mecanismos que esta actividad pide implementar, y demostrarlos exige un entorno donde tengamos el control.

## 2. Monitoreo del sistema

Se eligió **Netdata** porque muestra métricas por segundo sin configuración previa, descubre solo los contenedores de Docker y expone un panel web listo para capturar. Se levanta como un servicio más del stack:

**Listing 1:** `docker-compose.lab.yml` — servicio de monitoreo.

```

1 netdata:
2   image: netdata/netdata:stable
3   container_name: qc_netdata
4   hostname: questcash-lab
5   ports:
6     - "19999:19999"
7   cap_add:
8     - SYS_PTRACE           # leer metricas de otros procesos
9     - SYS_ADMIN
10  volumes:
11    - /proc:/host/proc:ro   # metricas del kernel del host
12    - /sys:/host/sys:ro
13    - /var/run/docker.sock:/var/run/docker.sock:ro # descubrir contenedores
```

Además, Nginx expone sus propias métricas para que Netdata las recoja (conexiones activas, peticiones totales, peticiones en espera):

**Listing 2:** `nginx.conf` — endpoint de métricas restringido a la red interna.

```

1 location = /nginx_status {
2   stub_status;
3   access_log off;
4   allow 127.0.0.1;
5   allow 172.16.0.0/12;   # red por defecto de Docker
6   deny all;
7 }
```

## 2.1 Procedimiento

1. Levantar el stack: `bash lab-semana3/scripts/00_arrancar.sh`
2. Generar carga real: `bash lab-semana3/scripts/03_generar_carga.sh`  
(8 clientes concurrentes durante 2 minutos contra el balanceador)
3. Con la carga corriendo, abrir `http://localhost:19999`
4. Capturar CPU, red, y la sección de contenedores Docker

**Un panel en reposo no es evidencia.** Gráficas planas demuestran que Netdata está instalado, no que el sistema esté siendo observado bajo actividad. Por eso las capturas se toman *mientras* corre el generador de carga, y con el selector de tiempo en «last 5 minutes» para que el pico se aprecie.

## 2.2 Evidencias a incorporar

CAPTURA M1

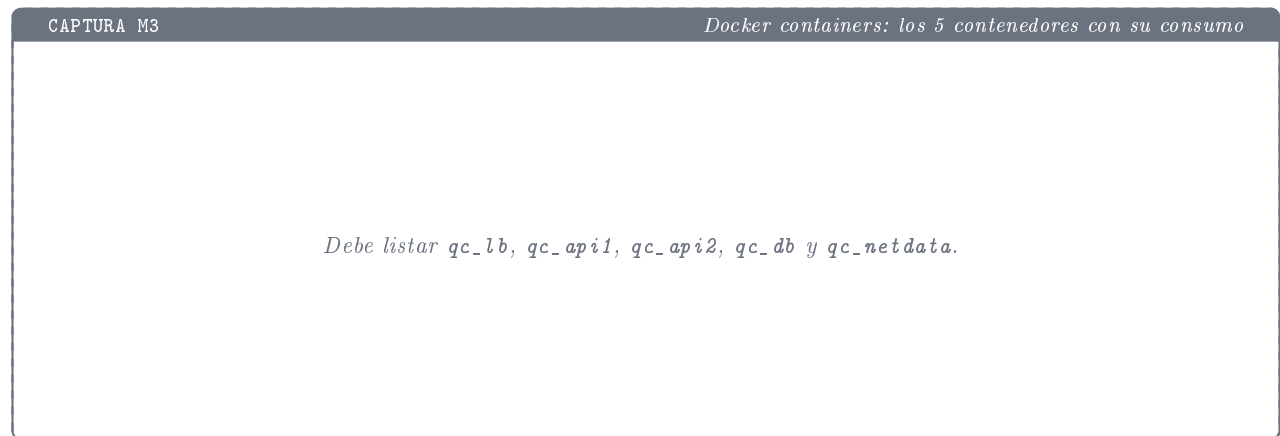
Vista general de Netdata con CPU y red en movimiento

*Panel principal de localhost:19999 durante la carga.  
Debe verse el reloj de la barra superior para fechar la evidencia.*

CAPTURA M2

System overview → CPU: el pico coincidiendo con la ráfaga

*Gráfica de utilización de CPU con el escalón provocado por los 8 clientes.*



El script `03_generar_carga.sh` complementa las capturas con métricas en texto: peticiones totales, *throughput* en req/s, la salida de `stub_status` y el consumo por contenedor según `docker stats`. Ese archivo (`evidencia/03_carga.txt`) sirve como respaldo numérico de lo que muestran las gráficas.

### 3. Firewall a nivel de sistema operativo

#### 3.1 Política

El firewall aplica el principio de mínimo privilegio a la red: **se deniega todo lo entrante** y se abre únicamente lo que el servicio necesita.

| Dirección | Política     | Razón                                                                                                        |
|-----------|--------------|--------------------------------------------------------------------------------------------------------------|
| Entrante  | <b>DENY</b>  | Nada entra salvo lo explícitamente permitido. Cada puerto abierto es una puerta más que alguien puede tocar. |
| Saliente  | <b>ALLOW</b> | El servidor necesita salir: actualizaciones del SO, DNS, renovación de certificados, APIs externas.          |

**Tabla 2:** Política por defecto de `ufw`.

#### 3.2 Puertos abiertos y por qué

| Puerto | Proto | Servicio | Justificación                                                                                                                                     |
|--------|-------|----------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| 22     | TCP   | SSH      | Única vía de administración. Se abre con <code>ufw limit</code> , que bloquea una IP tras 6 intentos en 30 s: frena la fuerza bruta automatizada. |
| 80     | TCP   | HTTP     | Necesario para el reto HTTP-01 de Let's Encrypt y para redirigir a HTTPS. No sirve contenido real.                                                |
| 443    | TCP   | HTTPS    | Por aquí entra <b>todo</b> el tráfico de usuarios. Es el único puerto que la aplicación realmente necesita.                                       |

**Tabla 3:** Puertos permitidos.

### 3.3 Puertos deliberadamente cerrados

Documentar lo que se cierra es tan importante como documentar lo que se abre: demuestra que la decisión fue consciente y no un descuido.

| Puerto | Servicio       | Por qué NO se abre                                                                                                                                    |
|--------|----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5432   | PostgreSQL     | Solo alcanzable desde la red interna de Docker. Una base de datos expuesta a internet es una de las causas más comunes de fuga de datos y ransomware. |
| 5000   | Flask/Gunicorn | Las instancias solo hablan con Nginx. Si fueran accesibles directo, se podría saltar el balanceador, el TLS y cualquier límite de tasa.               |
| 8080   | Nginx del lab  | Puerto de desarrollo. En un servidor real el proxy escucha en 80 y 443.                                                                               |
| 19999  | Netdata        | El panel expone topología, procesos y métricas: es reconocimiento gratis para un atacante. Se accede por túnel SSH.                                   |

**Tabla 4:** Puertos cerrados y su razón.

Para llegar a Netdata sin exponerlo: `ssh -L 19999:localhost:19999 usuario@servidor` y luego abrir `http://localhost:19999` en la máquina local. El puerto nunca queda publicado a internet.

### 3.4 Aplicación de las reglas

**Listing 3:** lab-semana3/firewall/ufw\_reglas.sh — extracto.

```

1  # Política por defecto
2  sudo ufw default deny incoming
3  sudo ufw default allow outgoing
4
5  # SSH con limite de tasa contra fuerza bruta
6  sudo ufw limit 22/tcp comment 'SSH - administracion remota (rate limited)'
7
8  # HTTP solo para redireccion y reto ACME
9  sudo ufw allow 80/tcp comment 'HTTP - redireccion a HTTPS y reto ACME'
10
11 # HTTPS: el trafico real de la aplicacion
12 sudo ufw allow 443/tcp comment 'HTTPS - trafico de la aplicacion'
13
14 sudo ufw --force enable
15 sudo ufw status verbose

```

**Advertencia importante sobre el entorno.** `ufw` es un envoltorio de `iptables`, que pertenece al kernel de **Linux**. macOS usa `pf`, que es otra cosa, y Docker Desktop ejecuta los contenedores dentro de su propia máquina virtual gestionando sus reglas de red automáticamente. Por lo tanto **este punto no se puede demostrar sobre macOS**.

Para presentar evidencia real hay dos caminos válidos: (a) una **VM Ubuntu en VirtualBox**, donde se ejecuta el script y se captura la salida; o (b) un **VPS** con IP pública, que permite además verificar desde fuera con `nmap` que solo responden 22, 80 y 443. La opción (b) es la más

contundente porque demuestra el firewall desde la perspectiva de un atacante y no solo desde la configuración declarada.

### 3.5 Evidencias a incorporar

CAPTURA F1

`sudo ufw status verbose` — política y reglas activas

*Debe verse **Status:** active, la línea  
Default: deny (incoming), allow (outgoing) y las tres reglas.*

CAPTURA F2

`sudo ss -tulnp | grep LISTEN` — qué escucha realmente

*Contrasta la configuración declarada con los procesos que de hecho  
tienen un socket abierto.*

CAPTURA F3

`nmap` desde otra máquina — la vista del atacante

*El escaneo solo debe encontrar 22, 80 y 443 abiertos.  
Opcional: requiere IP alcanzable desde otro equipo.*

## 4. Balanceador de carga con Nginx

Nginx funciona como *reverse proxy* y balanceador: el cliente se conecta a Nginx y este distribuye las peticiones entre dos instancias equivalentes de QuestCash. Ambas usan la misma imagen, la misma configuración y la misma base de datos.

### 4.1 El grupo de servidores

Listing 4: nginx.conf — definición del *upstream*.

```
1 upstream questcash_api {
2     server api1:5000 max_fails=1 fail_timeout=5s;
3     server api2:5000 max_fails=1 fail_timeout=5s;
4
5     keepalive 32;           # conexiones reutilizadas hacia el backend
6 }
```

Tres decisiones vale la pena explicar:

- **Round-robin** (el algoritmo por defecto): Nginx alterna entre los servidores de la lista. Se espera un reparto cercano al 50/50, que es justamente lo que mide la prueba 1.
- `max_fails=1 fail_timeout=5s`: al primer fallo, Nginx marca la instancia como caída y deja de mandarle tráfico durante 5 segundos; después la vuelve a probar sola. Este es el *health check pasivo*. Nginx de código abierto no hace sondeos activos —eso es Nginx Plus—, así que la detección ocurre al intentar usar el servidor.
- `keepalive 32`: mantiene conexiones abiertas hacia los backends en lugar de renegociar TCP en cada petición.

### 4.2 El reparto y la tolerancia a fallas

Listing 5: nginx.conf — bloque location principal.

```
1 location / {
2     proxy_pass http://questcash_api;
3
4     proxy_http_version 1.1;
5     proxy_set_header Connection "";
6     proxy_set_header Host      $host;
7     proxy_set_header X-Real-IP  $remote_addr;
8     proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
9     proxy_set_header X-Forwarded-Proto $scheme;
10
11     # Si la instancia elegida no responde, Nginx REINTENTA en la otra
12     # dentro de la MISMA petición: el cliente nunca ve el error.
13     proxy_next_upstream error timeout http_502 http_503 http_504;
14     proxy_next_upstream_tries 2;
15
16     proxy_connect_timeout 2s;
17 }
```

`proxy_next_upstream` es la pieza que convierte «dos servidores» en «alta disponibilidad». Sin ella, las peticiones que caigan en la instancia apagada devolverían **502** hasta que Nginx la marque como caída. Con ella, el reintento ocurre dentro de la misma petición y el resultado esperado de la prueba de failover es **100 % de disponibilidad**, no «unos cuantos errores al principio».



### 4.3 Cómo se sabe qué instancia respondió

Para poder *medir* el reparto, Nginx etiqueta cada respuesta y cada línea de su log con la instancia que la atendió:

**Listing 6:** nginx.conf — instrumentación para la evidencia.

```
1 log_format lb '$remote_addr [$time_local] "$request" '
2   'status=$status upstream=$upstream_addr '
3   'upstream_status=$upstream_status '
4   'rt=$request_time urt=$upstream_response_time';
5
6 # Cabecera de diagnostico (en produccion se quita: revela la topologia)
7 add_header X-Served-By $upstream_addr always;
```

### 4.4 Aislamiento de las instancias

En docker-compose.lab.yml, solo el balanceador publica un puerto. api1 y api2 no tienen sección ports:, así que son inalcanzables desde fuera de la red de Docker:

**Listing 7:** docker-compose.lab.yml — extracto de las instancias y el proxy.

```
1 api1:
2   build: { context: ., dockerfile: Dockerfile }
3   container_name: qc_api1
4   command: gunicorn --bind 0.0.0.0:5000 --workers 2 app:app
5   environment:
6     DATABASE_URL: postgresql://questcash:1234@db:5432/questcash
7     INSTANCE_ID: api1
8   depends_on:
9     db: { condition: service_healthy }
10  # Sin 'ports:' a proposito -> no accesible desde fuera
11
12 lb:
13   image: nginx:1.27-alpine
14   container_name: qc_lb
15   ports:
16     - "8080:80" # en produccion seria 80:80 y 443:443
17   volumes:
18     - ./lab-semana3/nginx/nginx.conf:/etc/nginx/nginx.conf:ro
19   depends_on: [api1, api2]
```

Se usa **Gunicorn** y no el servidor de desarrollo de Flask. El de Flask es monoproceso y no está pensado para concurrencia: con 8 clientes simultáneos los resultados de la prueba de carga no medirían el balanceo, medirían el cuello de botella del servidor de desarrollo.

### 4.5 Evidencias a incorporar

CAPTURA B1

docker compose ps — los 5 contenedores en Up

*Estado inicial: qc\_lb, qc\_api1, qc\_api2, qc\_db, qc\_netdata.*

CAPTURA B2

Salida de 01\_reparto.sh — distribución del tráfico

*Conteo de peticiones por instancia sobre 200 peticiones.  
Se espera un reparto cercano al 50/50 y 0 errores.*

## 5. Prueba de tolerancia a fallas

Esta es la evidencia central del entregable. El script `02_failover.sh` automatiza la prueba completa para que el resultado sea medible y no una impresión:

1. Lanza una ráfaga continua de peticiones al balanceador (una cada 0.5 s durante 45 s), registrando hora, código HTTP e instancia que respondió.
2. A los 15 s **apaga** `qc_api2` con el tráfico corriendo.
3. A los 30 s la vuelve a encender.
4. Al terminar, calcula disponibilidad, peticiones fallidas y reparto por instancia, y muestra cómo lo registró Nginx en su log de error.

**Listing 8:** `02_failover.sh` — núcleo de la prueba.

```

1  # Racha de peticiones en segundo plano
2  (
3      fin=$(( $(date +%s) + DURACION ))
4      while [ "$(date +%s)" -lt "$fin" ]; do
5          code=$(curl -s -m 5 -o /dev/null -D /tmp/qc_h2.txt \
6              -w "%{http_code}" "$BASE$RUTA")
7          inst=$(awk 'tolower($1)=="x-served-by:" {print $2}' /tmp/qc_h2.txt)
8          printf "%s HTTP=%s instancia=%s\n" "$(date +%H:%M:%S)" "$code" "$inst" >> "$LOG"
9          sleep 0.5
10     done
11 ) &
12
13 sleep $(( DURACION / 3 ))
14 docker stop "$VICTIMA"      # <-- se apaga una instancia EN VIVO
15 sleep $(( DURACION / 3 ))
16 docker start "$VICTIMA"     # <-- y se comprueba que Nginx la reincorpora

```

## 5.1 Qué se debe observar

| Métrica             | Resultado esperado y por qué                                                                                                                                  |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Disponibilidad      | <b>100 %</b> . Gracias a <code>proxy_next_upstream</code> , las peticiones que tocan la instancia caída se reintentan en la otra dentro de la misma petición. |
| Peticiones fallidas | <b>0</b> . Si aparecen 502 o timeouts, revisar que <code>proxy_next_upstream</code> esté activo.                                                              |
| Reparto             | ≈50/50 antes del apagón, <b>100 % en la sobreviviente</b> durante la caída, y vuelta al reparto tras reencender.                                              |
| Log de error Nginx  | Debe aparecer un mensaje de conexión rechazada hacia <code>api2</code> : es la prueba de que Nginx <i>detectó</i> la falla, no de que algo salió mal.         |

**Tabla 5:** Criterios de aceptación de la prueba de failover.

**Interpretación técnica.** Balancear no consiste en apagar frontend, API y base de datos entre sí —eso rompe el sistema y es el resultado esperado, no una prueba. La prueba correcta consiste en apagar **una copia del mismo servicio** que está detrás de Nginx. La otra copia permanece disponible y sigue accediendo al mismo PostgreSQL, que es lo que hace que el usuario no note nada.

## 5.2 Evidencias a incorporar

CAPTURA B3

Línea de tiempo del failover con la instancia ya detenida

*Debe verse la marca »> APAGANDO qc\_api2 y, en las líneas siguientes, HTTP=200 atendido íntegramente por qc\_api1.*

CAPTURA B4

Navegador cargando el sitio con una instancia caída

*http://localhost:8080/login respondiendo, junto a una terminal  
con docker compose ps mostrando qc\_api2 en Exited.*

CAPTURA B5

Resumen final: disponibilidad y peticiones fallidas

*El bloque RESUMEN del script, con la disponibilidad calculada.*

**Alternativa recomendada: video de 60 segundos.** El entregable acepta video, y para esta prueba es más convincente que las capturas porque muestra la continuidad. Basta grabar con **Cmd + Shift + 5** tres elementos visibles a la vez: la terminal corriendo el script, el navegador recargando el sitio, y otra terminal con **docker compose ps**.

## 6. Procedimiento de implementación

Todo el laboratorio vive en `questcash_web/lab-semana3/` y se ejecuta desde la raíz del proyecto.

Levantar el laboratorio y correr las pruebas

```
cd ~/questcash_web

# 1) Construir y levantar: Nginx + 2 instancias + Postgres + Netdata
bash lab-semana3/scripts/00_arrancar.sh

# 2) Comprobar que el trafico se reparte entre las dos instancias
bash lab-semana3/scripts/01_reparto.sh

# 3) PRUEBA CENTRAL: apagar una instancia con trafico en vivo
bash lab-semana3/scripts/02_failover.sh
```

```
# 4) Generar carga y capturar el monitoreo (abrir localhost:19999)
bash lab-semana3/scripts/03_generar_carga.sh

# Bajar todo al terminar
docker compose -f docker-compose.lab.yml down -v
```

Firewall - ejecutar en una VM Ubuntu o VPS, no en macOS

```
sudo bash lab-semana3/firewall/ufw_reglas.sh

# Comprobaciones que valen como evidencia
sudo ufw status verbose
sudo ufw status numbered
sudo ss -tulnp | grep LISTEN
nmap -Pn <ip-del-servidor>          # desde OTRA maquina
```

## 6.1 Estructura de archivos entregada

```
questcash_web/
  docker-compose.lab.yml      <- stack completo del laboratorio
  lab-semana3/
    README.md                 <- guia de uso
    CAPTURAS.md               <- que capturar y cuando
    nginx/nginx.conf          <- balanceador (upstream + failover)
    scripts/00_arrancar.sh     <- levanta y verifica el stack
    scripts/01_reparto.sh      <- mide la distribucion del trafico
    scripts/02_failover.sh     <- prueba de tolerancia a fallas
    scripts/03_generar_carga.sh <- carga sostenida para el monitoreo
    firewall/ufw_reglas.sh     <- reglas comentadas
    firewall/README-firewall.md <- justificacion puerto por puerto
    evidencia/                 <- salidas en texto de cada prueba
```

El script de arranque incluye un guardarraíl: si `.dockerignore` no excluye `data/raw/`, lo agrega. Esa carpeta contiene unos 750 MB de CSVs del ENIGH que la aplicación no usa en tiempo de ejecución —solo consume `data/processed/`— y sin excluirla Docker los copiaría al contexto de construcción en cada *build*.

## 7. Estado de las evidencias de la entrega

| Evidencia                       | Qué debe mostrar                                                                                    | Estado    | Dónde                  |
|---------------------------------|-----------------------------------------------------------------------------------------------------|-----------|------------------------|
| Configuración del balanceador   | <code>upstream</code> con 2 servidores, <code>round-robin</code> , <code>proxy_next_upstream</code> | ✓         | Anexo A                |
| Definición de las 2 instancias  | <code>api1</code> y <code>api2</code> sin puertos publicados                                        | ✓         | Anexo B                |
| Reglas de firewall documentadas | Puertos abiertos, cerrados y su justificación                                                       | ✓         | Sección 3, Anexo C     |
| Monitoreo con actividad real    | Netdata bajo carga: CPU, red, contenedores                                                          | pendiente | Capturas M1–M3         |
| <code>ufw</code> aplicado       | <code>ufw status verbose</code> en Linux                                                            | pendiente | Capturas F1–F3         |
| Reparto de carga medido         | Distribución sobre 200 peticiones                                                                   | pendiente | Captura B2             |
| Prueba de failover              | Instancia apagada y sitio respondiendo                                                              | pendiente | Capturas B3–B5 o video |

**Tabla 6:** Qué está listo y qué falta ejecutar.

**Nota de integridad académica.** Este documento no incluye capturas simuladas. Las configuraciones son archivos reales y ejecutables del proyecto; las evidencias visuales aparecen como recuadros vacíos hasta ser sustituidas por capturas obtenidas durante la ejecución real. Presentar un *mockup* como captura sería, además de deshonesto, inútil: no demuestra que el sistema funcione.

## 8. Conclusión

- Los tres mecanismos son complementarios, no independientes.** El monitoreo permite *observar* el estado del sistema; el firewall *reduce* la superficie de ataque; el balanceo *mantiene* la disponibilidad cuando una instancia falla. Ninguno sustituye a los otros.
- La arquitectura separa el punto de entrada público de las instancias internas.** Solo el balanceador publica un puerto; las instancias y PostgreSQL viven en la red interna de Docker. Esa separación es una medida de seguridad por sí misma, independiente del firewall del sistema operativo.
- La prueba de apagar una instancia es la evidencia principal de tolerancia a fallas,** y está automatizada para producir un número —la disponibilidad durante la caída— en lugar de una impresión.
- El entorno de producción impone límites reales.** Render en plan free no permite firewall a nivel SO, ni SSH, ni una segunda instancia. Reconocerlo y montar el laboratorio es la respuesta técnicamente correcta, no un atajo.

**Siguiente paso natural.** El laboratorio deja el balanceador escuchando en HTTP. El cierre lógico de esta línea de trabajo —y el pendiente que quedó señalado en el reporte de la semana 2— es terminar TLS en el propio Nginx: certificado de Let's Encrypt, redirección de 80 a 443 y cabecera HSTS. Con eso, el puerto 443 del firewall deja de ser una regla declarada y pasa a ser el camino real del tráfico.

## Anexo A — nginx.conf completo

```

1  # =====
2  # QuestCash -- Semana 3
3  # Nginx como balanceador de carga frente a 2 instancias idénticas de la API
4  # =====
5  worker_processes auto;
6
7  events {
8      worker_connections 1024;
9  }
10
11 http {
12     include      /etc/nginx/mime.types;
13     default_type  application/octet-stream;
14
15     # -----
16     # Formato de log ampliado: registra QUÉ instancia atendió cada
17     # petición ($upstream_addr) y con qué código respondió
18     # ($upstream_status). Es la evidencia del reparto y del failover.
19     # -----
20     log_format lb '$remote_addr [$time_local] "$request" '
21                  'status=$status upstream=$upstream_addr '
22                  'upstream_status=$upstream_status '
23                  'rt=$request_time urt=$upstream_response_time';
24
25     access_log /var/log/nginx/access.log lb;
26     error_log  /var/log/nginx/error.log warn;
27
28     sendfile    on;
29     tcp_nopush  on;
30     server_tokens off;          # no revelar la version de Nginx
31
32     # -----
33     # GRUPO DE SERVIDORES (upstream)
34     #
35     # Las dos instancias son IDENTICAS: misma imagen, mismo código,
36     # misma base de datos. Por eso una puede sustituir a la otra.
37     #
38     # Algoritmo: round-robin (el de por defecto). Nginx reparte las
39     # peticiones alternando entre los servidores de la lista.
40     #
41     # Health check PASIVO: si una instancia falla 'max_fails' veces,
42     # Nginx la marca como caída y deja de mandarle tráfico durante
43     # 'fail_timeout' segundos. Luego vuelve a probarla sola.
44     # -----
45     upstream questcash_api {
46         server api1:5000 max_fails=1 fail_timeout=5s;
47         server api2:5000 max_fails=1 fail_timeout=5s;
48
49         keepalive 32;          # conexiones reutilizadas hacia el backend
50     }
51
52     server {
53         listen 80;
54         server_name _;
55
56         # Cabecera de diagnostico: le dice al cliente que instancia respondió.
57         # (En producción se quita: da información de la topología interna.)
58         add_header X-Served-By $upstream_addr always;
59
60         # -----
61         # Salud del propio balanceador. Responde aunque las dos
62         # instancias estén caídas: distingue "se cayó Nginx" de
63         # "se cayeron los backends".
64         # -----

```



```

65     location = /lb-health {
66         access_log off;
67         add_header Content-Type text/plain;
68         return 200 "balanceador vivo\n";
69     }
70
71     # -----
72     # Métricas de Nginx para Netdata (conexiones activas, peticiones
73     # totales, etc.). Restringido a la red interna de Docker.
74     # -----
75     location = /nginx_status {
76         stub_status;
77         access_log off;
78         allow 127.0.0.1;
79         allow 172.16.0.0/12;    # red por defecto de Docker
80         allow 192.168.0.0/16;
81         deny all;
82     }
83
84     # -----
85     # Todo lo demas se reparte entre las dos instancias.
86     # -----
87     location / {
88         proxy_pass http://questcash_api;
89
90         proxy_http_version 1.1;
91         proxy_set_header Connection "";
92         proxy_set_header Host      $host;
93         proxy_set_header X-Real-IP  $remote_addr;
94         proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
95         proxy_set_header X-Forwarded-Proto $scheme;
96
97         # ---- Tolerancia a fallas -----
98         # Si la instancia elegida no responde, Nginx REINTENTA en la
99         # otra dentro de la MISMA petición. El cliente nunca ve el
100        # error: por eso la prueba de failover da 0 respuestas fallidas.
101        proxy_next_upstream      error timeout http_502 http_503 http_504;
102        proxy_next_upstream_tries 2;
103
104        proxy_connect_timeout 2s;
105        proxy_send_timeout 10s;
106        proxy_read_timeout 10s;
107    }
108 }
109 }

```

## Anexo B — docker-compose.lab.yml completo

```

1  # =====
2  # QuestCash -- Laboratorio de la semana 3
3  # Monitoreo + balanceo de carga + tolerancia a fallas
4  #
5  # Topologia:
6  #
7  #     cliente  ->  [ lb : Nginx :8080 ]
8  #                 |           |
9  #                 api1        api2      (2 instancias IDENTICAS)
10 #                 |           /
11 #                 [  db   ]      (Postgres, compartida)
12 #
13 #     [ netdata :19999 ] observa host + contenedores + Nginx
14 #
15 # Uso: docker compose -f docker-compose.lab.yml up -d --build
16 # =====
17 name: questcash-lab
18
19 services:
20
21 # -----
22 # Base de datos compartida por las dos instancias.
23 # Es lo que las hace intercambiables: mismo estado, mismo dato.
24 # -----
25 db:
26   image: postgres:15
27   container_name: qc_db
28   restart: unless-stopped
29   environment:
30     POSTGRES_USER: questcash
31     POSTGRES_PASSWORD: 1234
32     POSTGRES_DB: questcash
33   volumes:
34     - qc_pgdata:/var/lib/postgresql/data
35   healthcheck:
36     test: ["CMD-SHELL", "pg_isready -U questcash -d questcash"]
37     interval: 5s
38     timeout: 3s
39     retries: 10
40   networks: [qcnet]
41
42 # -----
43 # INSTANCIA 1 de la API. Se construye desde el Dockerfile del
44 # proyecto y se sirve con Gunicorn (no con el servidor de desarrollo
45 # de Flask, que es de un solo proceso y no aguanta concurrencia).
46 # -----
47 api1:
48   build:
49     context: .
50     dockerfile: Dockerfile
51   container_name: qc_api1
52   restart: unless-stopped
53   command: >
54     gunicorn --bind 0.0.0.0:5000
55             --workers 2
56             --access-logfile -
57             app:app
58   environment:
59     DATABASE_URL: postgresql://questcash:1234@db:5432/questcash
60     SECRET_KEY: ${SECRET_KEY:-lab-semana3-secret}
61     JWT_SECRET_KEY: ${JWT_SECRET_KEY:-lab-semana3-jwt-secret}
62     INSTANCE_ID: api1
63   depends_on:
64     db:

```

```

65     condition: service_healthy
66 networks: [qcnet]
67 # Sin 'ports:' a proposito -> las instancias NO son accesibles desde
68 # fuera. El unico punto de entrada es el balanceador.
69
70 # -----
71 # INSTANCIA 2. Identica a la 1 salvo el nombre. Ese es justamente
72 # el punto: son intercambiables.
73 # -----
74 api2:
75   build:
76     context: .
77     dockerfile: Dockerfile
78   container_name: qc_api2
79   restart: unless-stopped
80   command: >
81     gunicorn --bind 0.0.0.0:5000
82             --workers 2
83             --access-logfile -
84             app:app
85   environment:
86     DATABASE_URL: postgresql://questcash:1234@db:5432/questcash
87     SECRET_KEY: ${SECRET_KEY:-lab-semana3-secret}
88     JWT_SECRET_KEY: ${JWT_SECRET_KEY:-lab-semana3-jwt-secret}
89     INSTANCE_ID: api2
90   depends_on:
91     db:
92       condition: service_healthy
93   networks: [qcnet]
94
95 # -----
96 # BALANCEADOR DE CARGA. Unico contenedor con puerto publicado.
97 # -----
98 lb:
99   image: nginx:1.27-alpine
100  container_name: qc_lb
101  restart: unless-stopped
102  ports:
103    - "8080:80"          # en un servidor real seria 80:80 y 443:443
104  volumes:
105    - ./lab-semana3/nginx/nginx.conf:/etc/nginx/nginx.conf:ro
106    - qc_nginxlogs:/var/log/nginx
107  depends_on: [api1, api2]
108  networks: [qcnet]
109
110 # -----
111 # MONITOREO. Netdata observa CPU, memoria, red, disco y el estado
112 # de cada contenedor en tiempo real. Panel: http://localhost:19999
113 # -----
114 netdata:
115   image: netdata/netdata:stable
116   container_name: qc_netdata
117   hostname: questcash-lab
118   restart: unless-stopped
119   ports:
120     - "19999:19999"
121   cap_add:
122     - SYS_PTRACE
123     - SYS_ADMIN
124   security_opt:
125     - apparmor:unconfined
126   volumes:
127     - qc_netdataconfig:/etc/netdata
128     - qc_netdatalib:/var/lib/netdata
129     - qc_netdatacache:/var/cache/netdata
130     - /etc/passwd:/host/etc/passwd:ro
131     - /etc/group:/host/etc/group:ro

```

```
132     - /proc:/host/proc:ro
133     - /sys:/host/sys:ro
134     - /var/run/docker.sock:/var/run/docker.sock:ro
135     networks: [qcnet]
136
137 volumes:
138     qc_pgdata:
139     qc_nginxlogs:
140     qc_netdataconfig:
141     qc_netdatalib:
142     qc_netdatacache:
143
144 networks:
145     qcnet:
146         driver: bridge
```

## Anexo C — ufw\_reglas.sh completo

```

1  #!/usr/bin/env bash
2  # =====
3  # QuestCash -- Semana 3
4  # Firewall a nivel de sistema operativo (ufw / iptables)
5  #
6  # IMPORTANTE: este script es para un SERVIDOR LINUX (VM Ubuntu o VPS).
7  # macOS no usa ufw, y Docker Desktop corre sobre una VM propia, así que
8  # ufw no aplica ahí. Ver README-firewall.md para el detalle.
9  #
10 # Filosofía: DENEGAR TODO lo entrante por defecto y abrir únicamente
11 # lo que el servicio necesita para funcionar. Cada puerto abierto es
12 # una puerta mas que alguien puede tocar.
13 # =====
14 set -euo pipefail
15
16 if ! command -v ufw >/dev/null 2>&1; then
17     echo "ufw no está instalado. Instálalo con: sudo apt install ufw"
18     exit 1
19 fi
20
21 echo "==> Política por defecto"
22 # Todo lo que ENTRA se rechaza salvo que una regla lo permita.
23 sudo ufw default deny incoming
24 # Todo lo que SALE se permite: el servidor necesita actualizarse,
25 # resolver DNS y hablar con la base de datos o APIs externas.
26 sudo ufw default allow outgoing
27
28 echo "==> Reglas permitidas"
29
30 # --- SSH ---
31 # Sin esto te quedas fuera de tu propio servidor al activar el firewall.
32 # Se limita la tasa: ufw bloquea una IP que intente mas de 6 conexiones
33 # en 30 segundos, lo que frena los ataques de fuerza bruta.
34 sudo ufw limit 22/tcp comment 'SSH - administracion remota (rate limited)'
35
36 # --- HTTP ---
37 # Necesario para que Let's Encrypt valide el dominio y para redirigir
38 # a HTTPS. No sirve trafico real de la aplicacion.
39 sudo ufw allow 80/tcp comment 'HTTP - redireccion a HTTPS y reto ACME'
40
41 # --- HTTPS ---
42 # El puerto por el que entra TODO el trafico real de usuarios.
43 sudo ufw allow 443/tcp comment 'HTTPS - trafico de la aplicacion'
44
45 echo "==> Puertos que NO se abren (a proposito)"
46 cat <<'TXT'
47     5432 PostgreSQL    -> solo accesible desde la red interna de Docker.
48                        Exponerlo a internet es el error clasico que
49                        termina en base de datos secuestrada.
50     5000 Flask/API     -> las instancias solo hablan con Nginx, nunca
51                        directo con el exterior.
52     8080 Nginx (lab)   -> en produccion el proxy escucha en 80/443.
53     19999 Netdata      -> panel de monitoreo. Contiene informacion
54                        sensible de la infraestructura; se accede por
55                        tunel SSH: ssh -L 19999:localhost:19999 user@servidor
56 TXT
57
58 echo
59 echo "==> Activando el firewall"
60 sudo ufw --force enable
61
62 echo
63 echo "==> Estado final"
64 sudo ufw status verbose

```

```
65 echo
66 sudo ufw status numbered
```

### Fuentes de trabajo

- Repositorio web/backend: `questcash_web` — Flask, API `/api/v1`, PostgreSQL.
- Repositorio móvil: `questcash_mobile` — Expo / React Native, cliente de la API.
- Reporte de la semana 2: `docs/semana2/reporte_semana2.tex` — autenticación JWT.
- Actividad académica: EP.03.01 — Monitoreo, firewall y balanceador de carga.